05.2 — Post-Mortem Template + Example

> **For:** Documenting incidents after resolution. **Goal:** Learn from outages, prevent recurrence, build institutional memory. **Last updated:** 2026-08-29 **Pre-req reading:** 04.4 Incident Response Plan, 05.1 Change Management Policy, LEARNINGS.md

---

How to use this doc

This file has **two parts**:

- **Part 1 — Template:** copy-paste for any future incident
- **Part 2 — Example:** the 2026-08-29 500-outage, filled in with real data

A post-mortem is **mandatory** for SEV-1 + SEV-2 incidents (within 24h of resolution). Optional for SEV-3/4.

Part 1 — Template

Copy this section into a new file: `docs/phase6/post-mortems/YYYY-MM-DD-{slug}.md`

Incident summary

One paragraph (3-5 sentences). What happened, who was affected, how it was resolved.

```
On {date} at {time} IST, {env} experienced {symptom}.
The root cause was {root cause}.
{Number of users} were affected for {duration}, with {data loss Y/N}.
Resolution: {fix applied}.
This post-mortem documents the timeline, root cause, lessons learned, and preventive measures.
```

Timeline

All times in UTC + IST (Venkat is IST UTC+5:30).

UTC	IST	Event
yyyy-mm-dd HH:MM	*yyyy-mm-dd HH:MM*	*description*
…	…	…

Typical events to include: - Detection (when first noticed) - Triage start (when investigation began) - Hypothesis formed - Verification (parent-verify, subagent spawned, etc.) - Venkat notified (if applicable) - Fix applied - Verification of fix - All-clear communicated - Post-mortem started

---

Root cause

Technical cause: *what specifically broke (e.g., gunicorn –preload sys.path freeze after install-app).*

Contributing factors: - *factor 1 (e.g., no automated restart in deploy script)* - *factor 2 (e.g., MEMORY.md password drift)*

Why wasn't this caught earlier: - *reason 1 (e.g., install-app on running container with –preload gunicorn is rare)* - *reason 2 (e.g., no smoke test immediately after install-app)*

Impact

Metric	Value
Users affected	*% or count*
Downtime	*HH:MM*
Requests failed	*count or "100% during window"*
Data loss	*Y / N, scope*
Revenue impact	*estimate, if any*
Reputation impact	*qualitative*

Resolution

What fixed it:

```
{commands}
```

Steps: 1. *{step 1}* 2. *{step 2}* 3. *{step 3}*

Side effects of fix: *any unintended consequences, e.g., prod login 401 after restart due to password drift.*

Lessons learned

New lessons for LEARNINGS.md (numbered, prefixed #1XX):

Lesson #XXX: {title} ({date})

Body: *symptom + fix + when to apply.*

Lesson #XXX: {title} ({date})

Body.

Reinforced lessons: *existing lessons that this incident validated (e.g., #72 parent-verify).*

Action items

Preventive measures. Each item: owner + due date + status.

```
- [ ] {action item 1}    owner={name}    due={date}
- [ ] {action item 2}    owner={name}    due={date}
- [x] {action item 3}    owner={name}    status=done
```

Categories: - **Immediate** (done within 24h): update LEARNINGS.md, update runbook - **Short-term** (done within 1 week): automate restart, add monitoring - **Long-term** (done within 1 quarter): architecture change, new tool

---

References

- Commits: `{hash}` …
- PRs: #{number} …
- Subagent RunIds: `{name}` …
- Memory log: `memory/YYYY-MM-DD.md` §{section}
- Related lessons: #{number}, #{number}
- Related runbooks: [04.X …], [05.X …]

Part 2 — Example — 2026-08-29 500-outage post-mortem

The canonical example. This is the actual incident from `memory/2026-08-29.md`, documented properly per this template.

Incident summary

On **2026-08-29 at 03:06 IST**, both `pberpprod` and `pberpdev` environments returned HTTP 500 on every web request. Root cause: gunicorn's `--preload` flag froze `sys.path` at container startup (dev=9 days ago, prod=2 days ago), and today's `haritha_hospital` install wrote a `.pth` file that was invisible to gunicorn's process tree. **100% of web requests failed for ~10 minutes on both prod and dev**. Zero data loss. Resolution: `docker restart erp-{dev,prod}-backend-1` (fresh gunicorn = fresh sys.path).

Timeline

All times IST (UTC+5:30). UTC included for cross-region debugging.

UTC	IST	Event
2026-08-28 ~16:00	2026-08-28 21:30	`haritha_hospital` install on pberpprod
2026-08-28 ~17:30	2026-08-28 23:00	`haritha_hospital` install on pberpdev
2026-08-28 21:36	2026-08-29 03:06	Venkat reports both envs down via Telegram

UTC	IST	Event
2026-08-28 21:38	2026-08-29 03:08	Main session begins triage; spawns `haritha-diag-500` subagent
2026-08-28 21:40	2026-08-29 03:10	Subagent identifies root cause (gunicorn –preload + .pth file missing)
2026-08-28 21:41	2026-08-29 03:11	Main session asks Venkat to verify; Venkat asks “will restart clear the data?”
2026-08-28 21:42	2026-08-29 03:12	Spawns `haritha-verify-volumes` subagent
2026-08-28 21:44	2026-08-29 03:14	Volume verify proves all data on named volumes (no ephemeral layer)
2026-08-28 21:45	2026-08-29 03:15	Venkat says YES, restart approved
2026-08-28 21:46	2026-08-29 03:16	`docker restart erp-dev-backend-1` (30s downtime)
2026-08-28 21:46	2026-08-29 03:16	dev curl → 200, login → 200 □
2026-08-28 21:47	2026-08-29 03:17	`docker restart erp-prod-backend-1` (30s downtime)
2026-08-28 21:47	2026-08-29 03:17	prod curl → 200 □, login → **401** ⚠
2026-08-28 21:50	2026-08-29 03:20	Prod login 401 investigated: MEMORY password drifted (Lesson #154)
2026-08-28 22:00	2026-08-29 03:30	Prod password resolved by retrieving from container `.env`
2026-08-28 22:30	2026-08-29 04:00	Post-mortem started; LEARNINGS #153 + #154 added

---

Root cause

Technical cause

Gunicorn runs as **PID 1** in the backend container, with the **`--preload`** flag. This means:

1. At container startup, gunicorn imports the app code **once** (in the master process).
2. After import, gunicorn forks worker processes.
3. **`sys.path` is frozen at the master’s import time.** Subsequent changes (e.g., new `.pth` files added by `pip install` or `bench install-app`) are invisible to gunicorn’s process tree.

When `haritha_hospital` was installed today via `bench install-app`, Frappe wrote a `.pth` file pointing to the new app directory. But:

- The container's Python interpreter saw the `.pth` file (a fresh `python -c "import haritha_hospital"` worked).
- Gunicorn's **frozen** `sys.path` did NOT include it.

Result: every worker process → `import haritha_hospital` → `ModuleNotFoundError` → HTTP 500.

Why today specifically?

The bug always existed (gunicorn –preload sys.path is always frozen), but today was the first time we installed a NEW app on a RUNNING container with –preload gunicorn. Previous installs were either:

- On a freshly-built container (gunicorn started AFTER install)
- Same app re-installed (already in sys.path)

Contributing factors

- **No automated restart in deploy script.** The deploy script did `bench install-app` but didn't `docker restart`. Lesson #153 documents this gap.
- **MEMORY.md password drift** (Lesson #154). The prod login 401 immediately after restart was a SECONDARY failure — the fix worked, but the password literal in MEMORY had drifted. This delayed full recovery by ~13 min.
- **No monitoring alert** for HTTP 500 spike. Detection was via Venkat manually trying to use the system.

Why wasn't this caught earlier?

- **Install-app on running container is rare.** Most app installs happen during container build.
- **No smoke test in deploy script** post-install-app. A simple curl after install would have caught it.
- **No automated restart step.** The deploy script ended after `bench install-app` + `bench migrate`, but didn't include the required restart.

Impact

Metric	Value
Users affected	100% of web requests on pberpprod + pberpdev
Downtime	~10 minutes (03:06 → 03:17 IST, with restart sequence)
Total downtime (env-by-env)	~30-60s per env (just the restart time)
Requests failed	every request in the 10-min window × both envs
Data loss	**ZERO** — all data on named Docker volumes, preserved across restart
Revenue impact	none (no paying users on these envs at 3 AM)
Reputation impact	none externally (Venkat was the only one who noticed)
Subagent cost	3 subagents spawned (~30k tokens)

Metric	Value
Lesson value	+2 new lessons (LEARNINGS #153, #154), +1 reinforced (#72)

Resolution

What fixed it

```
docker restart erp-dev-backend-1
sleep 45
docker restart erp-prod-backend-1
sleep 45
```

Steps

1. **Confirmed outage** (curl from outside, both envs HTTP 500)
2. **Checked containers** (all "Up", not infra-level failure)
3. **Hypothesized** (Lesson #46 + install-app on running container = stale sys.path)
4. **Verified root cause** (direct `python -c "import"` works, gunicorn traceback says ModuleNotFoundError)
5. **Volume-verified data persistence** (Lesson #72 — spawned subagent to confirm data safety)
6. **Restarted dev first** (lower risk, validates fix)
7. **Restarted prod** (after dev proven working)
8. **Resolved secondary issue** (prod login 401 → retrieved actual password from container `.env`)

Side effects of fix

- **Prod login 401 immediately after restart.** This was NOT a fix failure — Frappe was responsive (200 = haritha loaded), but the admin password literal in MEMORY.md had drifted. Fix: retrieved actual password from `docker exec erp-prod-backend-1 printenv ADMIN_PASSWORD` (or similar env var). This triggered Lesson #154 (verify MEMORY passwords quarterly).
- **No data loss.** Named volumes preserved across restart (Lesson #154: `docker restart` ≠ `docker-compose down`).

Lessons learned

NEW Lesson #153 — Gunicorn `--preload` + new Python package = backend container restart required (2026-08-29)

Gunicorn PID 1 with `--preload` flag imports app code once at startup, freezes `sys.path`, then forks workers. New packages installed AFTER startup are invisible to gunicorn process tree even if `.pth` files are written.

Symptom: Every request returns `ModuleNotFoundError: No module named '<new_app>'` → HTTP 500. Direct `python -c "import <new_app>"` in fresh shell succeeds.

Fix: `docker restart erp-{env}-backend-1` (NOT just `bench restart` — that doesn't restart gunicorn). After restart, gunicorn re-imports with fresh sys.path.

CRITICAL: After ANY `bench install-app` on a running container, ALWAYS restart the backend container.

NEW Lesson #154 — `docker restart` ≠ `docker-compose down` (2026-08-29)

`docker restart`: stops + starts container, **volumes untouched** (named + anonymous both preserved).

`docker-compose down`: stops + removes containers. **Named volumes preserved, anonymous volumes preserved**.

`docker-compose down -v`: stops + removes containers **AND named volumes**. ALL DATA LOST.

Use `restart` for in-place recovery. NEVER use `down -v` unless wiping.

REINFORCED Lesson #72 — Parent-verify before claiming SUCCESS (2026-08-29)

When fixing a container issue, **always verify volume persistence before restart**. In this incident:

- Venkat asked "will restart clear the data?" (legitimate concern)
- Spawned volume-verify subagent
- Confirmed all data on named volumes (NOT ephemeral)
- Restart proceeded with confidence

Without Lesson #72, we might have used `docker-compose down` (which preserves named volumes but kills all sessions) or worse `down -v` (which deletes everything). Parent-verify saved potential data loss.

REINFORCED Lesson #46 — After `install-app`, restart backend AND workers

This lesson EXISTED but was for the older pattern. Today's incident is a sharper version: `install-app` on a `--preload` gunicorn container specifically needs backend container restart, not just `bench restart`. Lesson #153 supersedes #46 for this specific case.

Action items

Immediate (done within 24h)

- [x] Add LEARNINGS.md #153 (done 2026-08-29 03:30 IST)
- [x] Add LEARNINGS.md #154 (done 2026-08-29 03:35 IST)
- [x] Document gunicorn restart in deployment runbook (04.1) (done 2026-08-29 04:00 IST)
- [x] Add case study to incident response runbook (04.4) (done 2026-08-29 04:00 IST)
- [x] Update MEMORY.md with verified DB passwords (done 2026-08-29 03:30 IST)
- [x] Update MEMORY.md "Critical Lessons" section with new lessons (done 2026-08-29 03:35 IST)

Short-term (done within 1 week)

- [ ] Add automated `docker restart` step to deploy script (target: 2026-09-05)
- [ ] Add HTTP 500 monitor (Telegram alert on `tabError Log` spike) (target: 2026-09-05)

- ☐ Harden `dev_backup.sh` + `qa_backup.sh` + `deverp_backup.sh` per Lesson #79 (target: 2026-09-05)

Long-term (done within 1 quarter)

- ☐ Consider periodic restart check (cron?) — low priority, monitor first (target: 2026-12-01)
- ☐ Add Prometheus + Grafana for container metrics (deferred — manual cadence OK for now)

Done as part of this incident

- ☒ Volume-verify subagent spawned (Lesson #72 in action)
- ☒ Parent-verify (Lesson #72 in action)
- ☒ Restart order: dev first, then prod (risk-minimized approach)
- ☒ Secondary issue resolved (MEMORY password drift, Lesson #154)
- ☒ Post-mortem started within 24h of resolution

References

Commits

- `4023ccde84d1fb0600ab1552e81076edea5d32ca` — haritha_hospital skeleton (pushed 2026-08-29, ~02:30 IST)
- `93e8c482357b747a5237ad1f2b0270216943b300` — fixtures + hooks.py (pushed 2026-08-29, ~02:45 IST)

Subagent RunIds (chronological)

- `haritha-diag-500` — diagnosis (premium Minimax) — root cause: gunicorn –preload
- `haritha-verify-volumes` — volume persistence check (Lesson #72)
- `haritha-restart-both` — restart execution + verify

Memory logs

- `memory/2026-08-29.md` §"CRITICAL INCIDENT: Both envs 500'd (2026-08-29 ~03:06 IST)"

LEARNINGS

- NEW: #153, #154
- REINFORCED: #46, #72
- RELATED: #48 (8-phase test), #79 (backup hardening), #151 (fixtures declaration)

Runbooks updated

- 04.1 Deployment Runbook — added "docker restart" step
- 04.4 Incident Response Plan — added case study
- 05.1 Change Management Policy — added Lesson #153 to anti-patterns

MEMORY.md

- Critical Lessons section updated with #153, #154
- DB passwords verified + corrected (2026-08-29 prod password corrected)

Five Whys (analysis technique)

For deeper root-cause analysis, use "Five Whys" on the primary cause:

Q1: Why did both envs return HTTP 500? A: Gunicorn couldn't import the newly-installed `haritha_hospital` app.

Q2: Why couldn't gunicorn import it? A: Gunicorn's `--preload` flag froze `sys.path` at startup; the new `.pth` file was not visible.

Q3: Why didn't the deploy script restart gunicorn? A: The deploy script ended after `bench install-app` + `bench migrate`, but didn't include a restart step. The Lesson #46 recommendation was for `bench restart`, which doesn't restart gunicorn PID 1.

Q4: Why didn't anyone catch the gap between `bench restart` and gunicorn restart? A: We've never installed a NEW app on a RUNNING container with `--preload` gunicorn before. All previous installs were on freshly-built containers where gunicorn started AFTER install.

Q5: Why was there no smoke test in the deploy script? A: Manual deploy process; no automation. Smoke test was expected to be "manual curl after deploy", but happened in this case 8 hours later (Venkat's morning check).

Counter-measure: add automated `docker restart` + curl smoke test to deploy script. (See action items, short-term.)

Every incident is a lesson. Document it or repeat it.